

Discovery Executive Summary

Project: discovery-20jul-2026 · Generated: 20/07/2026, 18:37:57

EXECUTIVE SUMMARY

This report consolidates the overall ratings, key findings, and recommended actions from the 2 discovery analyses run across this codebase (frontend and backend). Each section below reproduces that analysis's executive view; full evidence and diagrams live in the individual reports.

Portfolio Overview

#	Analysis	Overall Rating	Hotspot Score
1	Architecture & Design Analysis	HIGH RISK	—
2	Frontend Modernization Analysis	HIGH RISK	—

1. Architecture & Design Analysis

OVERALL CODEBASE RATING — ARCHITECTURE & DESIGN

High Risk

Driven by High-Risk Missing Service Layer / Missing Repository Pattern and Moderate cross-domain boundary violations.

EXECUTIVE SUMMARY

The codebase is a two-layer system: a Laravel API backend and a React/Vite frontend. The backend is structurally thin at the controller level, but the real risk comes from widespread direct model access in handlers and the repeated reuse of domain models across connect, discovery, and legacy reporting flows. On the frontend, HTTP access is centralized through `frontend/src/api/client.ts`, but the page layer still carries workflow logic and several legacy/stateful patterns. The most important architectural pressure points are H2/H3 on the backend and F1/F5 on the frontend, with H8 adding cross-domain coupling risk.

1.1 Benchmark Ratings Summary

#	Hotspot	Primary KPI	GOOD	MODERATE	HIGH RISK	Measured	Rating
---	---------	-------------	------	----------	-----------	----------	--------

H1	Fat Controllers	Avg LOC per controller	<150	150–300	>300	54 LOC avg across 6 controllers	GOOD
H2	Missing Service Layer	Controllers accessing repos/models	<10	10–20	>20	23 direct model access points in controllers	HIGH P
H3	Missing Repository Pattern	Direct DB access points	<10	10–20	>20	23 direct ORM/data-access points outside repositories	HIGH P
H4	Circular Dependencies	Dependency cycles	0	1–3	>3	Not observed from sampled dependency graph	GOOD
H5	Shared Utility Abuse	Utility files w/ business logic	0	1–5	>5	1 legacy mapper with business-shaped transformation logic	MODE
H6	Direct SQL in Controllers	ORM compliance %	>90%	60–90%	<60%	100% of sampled controller queries use ORM/Eloquent, not raw SQL	GOOD
H7	God Classes	Classes >1000 LOC	0	1–3	>3	0 classes above 1000 LOC	GOOD
H8	Domain Boundary Violations	Cross-domain access points	0	1–5	>5	4 cross-domain accesses in legacy reporting / realtime flows	MODE
H9	Shared Database Coupling	Tables shared across domains	<10%	10–30%	>30%	0 shared tables observed; domains use separate model sets	GOOD
F1	Business Logic in Components	Avg LOC per component	<150	150–300	>300	96 LOC avg across 10 frontend components	GOOD
F2	Missing Frontend Service/Data Layer	Components w/ inline API calls	<10	10–20	>20	0 components with hard-coded inline fetch/axios URLs; all calls flow through <code>api/client.ts</code>	GOOD
F3	God / Oversized Components	Components >400 LOC	0	1–3	>3	0 components above 400 LOC	GOOD
F4	Prop Drilling / Global State Abuse	Max prop-drilling depth	≤2	3–4	>4	1-level router/layout composition; no deep prop chains observed	GOOD
F5	Legacy / Inconsistent Component Patterns	Legacy-pattern components	0	1–10	>10	2 legacy patterns observed (<code>LegacyMonitorPoller.jsx</code> , <code>useEffect</code> polling widget)	MODE

1.4 Actions Required

Hotspot	Action	Rating	Priority
H2 Missing Service Layer	Extract application services for connect/discovery workflows and move reachability + tree-building logic out of controllers and <code>RealTimeTestService</code> .	HIGH RISK	CRITICAL
H3 Missing Repository Pattern	Introduce repositories for the four Eloquent model groups and stop issuing direct model queries from controllers/services.	HIGH RISK	CRITICAL
H5 Shared Utility Abuse	Split <code>LegacyDataMapper</code> into domain-owned mappers and remove <code>extract()</code> -based mapping.	MODERATE	MEDIUM
H8 Domain Boundary Violations	Define explicit connect, discovery, and reporting boundaries; move cross-domain reads behind projections or interfaces.	MODERATE	HIGH
F5 Legacy / Inconsistent Component Patterns	Convert the remaining class/polling legacy UI patterns to hooks and shared query-driven components.	MODERATE	MEDIUM

1.5 Expected Outcomes

- Controllers stay thin while business workflows move into reusable services.
- Repositories create a single persistence seam for connect, discovery, and reporting data.
- Legacy reporting becomes easier to test and safer to change without affecting live workflows.
- Frontend pages keep using the shared API client while moving complex orchestration into hooks.
- Connect and discovery can evolve as clearer bounded contexts with lower change amplification.

2. Frontend Modernization Analysis

OVERALL CODEBASE RATING — FRONTEND MODERNIZATION

High Risk

The largest driver is massive component scale, with `App.jsx`, `Dashboard.jsx`, `StepFlowSelection.jsx`, and `StepIdeConfig.jsx` each carrying too much routing, orchestration, and form logic in one place.

EXECUTIVE SUMMARY

This workspace has a mature React frontend, and the dominant idiom is already function components with hooks on top of React 19.2.5. The main modernization gap is not framework migration; it is component scale and orchestration complexity, especially in `App.jsx`, `Dashboard.jsx`, and the setup flow screens. Legacy class-based UI is effectively absent aside from a single error boundary, so the codebase is mostly aligned with current React patterns. The most material risks are oversized components, repeated imperative state synchronization, and deep feature components that mix routing, auth,

layout, and data orchestration. Overall, the codebase is usable and mostly modern, but the largest screens would benefit from extracting shared shell, state, and form primitives.